Assignment No: 8

Title: Create a basic Single Page Application using React with functional components and JSX.

Aim: To create a basic Single Page Application (SPA) using ReactJS, built entirely from functional components and JSX, in which navigating between views re-renders the page without any full browser reload.

Objectives:

1. To understand what a Single Page Application is and how it differs from a multi-page website.
2. To set up a React project using the Vite build tool.
3. To understand the role of the single root HTML element and createRoot.
4. To write functional components and compose them into a component tree.
5. To write JSX and understand how it compiles to React.createElement calls.
6. To switch between views in a single page without a browser reload.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- Node.js (v18 or later) and npm
- Vite build tool
- React 19 and React DOM
- React Developer Tools browser extension

Theory:

1. Single Page Application (SPA)

A Single Page Application loads one HTML document once. All further navigation replaces part of the DOM with JavaScript instead of requesting a new document from the server. This gives a faster, application-like experience because only data, not a whole page, crosses the network after the first load.

Aspect	Multi-Page Application	Single Page Application
Navigation	Full HTTP request per page	DOM re-render in the browser
Page reload	Yes, on every link	No
State	Lost on each navigation	Preserved in memory

Initial load	Light	Heavier (the bundle)

2. Introduction to ReactJS

React is a JavaScript library for building user interfaces from reusable components. React keeps a virtual representation of the DOM, compares it with the previous version when state changes, and applies only the differences to the real DOM. This process is called reconciliation.

3. Functional Components

A functional component is a plain JavaScript function whose name starts with a capital letter and which returns JSX. It receives its inputs as a single props object. Functional components replaced class components as the standard style once hooks were introduced in React 16.8.

Example:

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

// or with an arrow function and destructuring
const Welcome = ({ name }) => <h1>Hello, {name}</h1>;
```

4. JSX

JSX is a syntax extension that lets XML-like markup be written inside JavaScript. It is not understood by the browser; the build tool compiles it into function calls.

JSX and its compiled form:

```
// written
<h1 className="title">Hello</h1>

// compiled
React.createElement('h1', { className: 'title' }, 'Hello')
```

- Attributes use camelCase: className instead of class, onClick instead of onclick.
- A JavaScript expression is embedded with curly braces: {value}.
- A component must return exactly one root node; <>...</> is an empty wrapper called a Fragment.
- Every element in a list needs a stable key prop.

5. Mounting the Application

index.html contains a single empty container, usually . The entry module obtains that node, calls createRoot on it and renders the root component into it. StrictMode is a development-only wrapper that double-invokes render to surface unsafe side effects.

6. Project Structure Created by Vite

```
lipisutra/
  index.html            <- the single HTML document
  package.json          <- dependencies and scripts
  vite.config.js        <- build configuration
  src/
    main.jsx            <- entry point, mounts React
    App.jsx             <- root component
    index.css           <- global stylesheet
    components/         <- reusable functional components
    services/           <- plain JS modules (API calls)
```

Course Project Implementation:

Course-Project Mapping: Fully available in the course project. LipiSutra is itself a Single Page Application: a single index.html, a createRoot mount in src/main.jsx, and a root App component that swaps between three views without a reload. Submit index.html, src/main.jsx and src/App.jsx, with src/components/Navbar.jsx as the child component that triggers the view change.

SPA element	File	Evidence
Single HTML document	index.html	plus one module script
React mount	src/main.jsx	createRoot(document.getElementById('root')).render(...)
Root functional component	src/App.jsx	export default function App()
JSX view switching	src/App.jsx	{view === "public" && <PublicView />} - no reload, no router needed
Child functional components	src/components/*.jsx	Navbar, PublicView, HistorianView, MuseumView, ResultCard

Program:

Project file - index.html (the one and only HTML document)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <title>lipisutra</title>
  </head>
  <body>
    <!-- the single mount point : React owns everything inside it -->
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Project file - src/main.jsx (entry point)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Project file - src/App.jsx (root functional component, abridged)

```
import { useState, useEffect } from "react";
```

```
import Navbar        from "./components/Navbar";
import PublicView    from "./components/PublicView";
import HistorianView from "./components/HistorianView";
import MuseumView    from "./components/MuseumView";
import backImg       from "./assets/back.jpg";

const DARK  = { name: "DARK",  bg: "#1a0f00", surface: "#2a1800", border: "#8B6914",
                accent: "#D4A017", text: "#f5e6c8", subtext: "#c9a96e" };
const LIGHT = { name: "LIGHT", bg: "#f5ede0", surface: "#ede0cc", border: "#8B4513",
                accent: "#6B2D0E", text: "#1a0800", subtext: "#5c3317" };

export default function App() {
  const [view, setView]   = useState("public");     // "public" | "historian" |
"museum"
  const [page, setPage]   = useState("home");
  const [themeMode, setThemeMode] = useState(
    () => localStorage.getItem("themePreference") || "DARK"
  );

  useEffect(() => {
    localStorage.setItem("themePreference", themeMode);
  }, [themeMode]);

  const theme = themeMode === "DARK" ? DARK : LIGHT;
  const toggleTheme = () => setThemeMode(prev => (prev === "DARK" ? "LIGHT" :
"DARK"));

  return (
    <div className="min-h-screen relative flex flex-col font-sans"
         style={{ backgroundColor: theme.bg, color: theme.text,
                  transition: "background 0.3s, color 0.3s" }}>

      <header className="absolute top-4 right-4 z-[100]">
        <button onClick={toggleTheme}>
          {themeMode === "DARK" ? "Dark" : "Light"}
        </button>
      </header>

      {/* a child functional component receiving props */}
      <Navbar
        currentView={view}  onViewChange={setView}
        currentPage={page}  onPageChange={setPage}
        theme={theme}
      />

      {/* THE SPA SWITCH : the URL never changes, the page never reloads */}
      <main className="flex-1 overflow-x-hidden relative z-10 w-full">
        <>
          {view === "public"    && <PublicView    theme={theme} />}
          {view === "historian" && <HistorianView theme={theme} />}
          {view === "museum"    && <MuseumView    theme={theme} />}
        </>
      </main>

    </div>
  );
}
```

Commands used to create and run the project

```
# create the project
npm create vite@latest lipisutra -- --template react
cd lipisutra

# install dependencies
npm install
```

```
# start the development server (http://localhost:5173)
npm run dev

# produce the production bundle in dist/
npm run build
```

Output:

- npm run dev starts the Vite server and the application opens at http://localhost:5173 with the Public / Student view rendered.
- Clicking the Historian or Museum Admin role button replaces the entire main region instantly; the browser tab shows no reload spinner and the Network tab records no document request.
- The chosen theme survives every view change because the state lives in the App component in memory, which a multi-page site could not do.
- React Developer Tools shows the component tree App > Navbar + PublicView > UploadSection + MapSection, confirming that every node is a functional component.
- Inspecting the page shows that all of the rendered markup lives inside the single element from index.html.

Conclusion:

In this assignment, we created a basic Single Page Application using React with functional components and JSX. We understood how a single HTML document is mounted with createRoot, how JSX compiles to React.createElement calls, and how conditional rendering lets three complete views share one page without a browser reload. The LipiSutra course project is itself exactly such an SPA, so index.html, src/main.jsx and src/App.jsx were submitted directly as the implementation.

References:

7. React Official Documentation, https://react.dev/learn
8. React - Built-in Hooks Reference, https://react.dev/reference/react/hooks
9. Vite Guide, https://vite.dev/guide/